



computational morphology

LING 430 Computational Linguistics Fall 2026

lecture 5a

Sep 22 2026

Siyu Liang

siyu.liang@rice.edu

logistics

- grades for hw1 and hw2 released



last time

- encoding: Unicode, code points, UTF-8
- tokenization
- byte-pair encoding



agenda

- morphology
- text normalization



today's notebook

class05a.ipynb

- course website → week 5 → activity
- **File → Save a copy in Drive**



morphology

morphology

- concepts we've introduced so far
 - string
 - token
 - sub-token
- but natural language is built on *words*, which concerns the field of morphology

morphology

- a **morpheme** is a **minimal meaning-bearing unit**
 - *fox*: one morpheme
 - *cats*: two morphemes, *cat* and the plural –s
- **morphology** is the study of how words are built out of them
- morphemes and words don't necessarily match

Emma work-ed care-ful-ly wash-ing the glass-es

morphology

- **interlinear glossed text (IGT)** is the standard way to show morphemes
 - 1. text segmented into morphemes
 - 2. aligned glosses
 - 3. free translation
- hyphens mark morpheme boundaries inside a word, spaces mark words, function morphemes marked with abbreviations
 - for more, see [the Leipzig Glossing Rules](#)

Emma work-ed care-ful-ly wash-ing the glass-es

Emma work-PST care-ADJ-ADV wash-PROG DEF glass-PL

‘Emma worked carefully washing the glasses.’

morphology

- different languages can have different morphologies
- “梅干菜用清水泡软捞出后沥干切碎”
- in Chinese, no space is written to separate words
 - each character is a morpheme
 - words are usually formed through **compounding** not **affixation**



梅-干-菜	用	清-水	泡	软	捞-出	后	沥-干	切-碎
plum-dry-vegetable	use	clear-water	soak	soft	remove-out	after	drip-dry	chop-fragment

‘Soak the preserved vegetable in water until soft, remove, drain, and chop.’

morphology

roots & affixes

	examples	what it does
root	work, glass, care	the main morpheme carrying meaning
inflectional affix	-s plural, -ed past, -ing	grammatical, productive, predictable; the part of speech is usually unchanged
derivational affix	-ful, -ly, un-	makes a new word, often a new part of speech: care → careful → carefully

morphology

- a **clitic** acts like a word in the syntax but is reduced in form and attached to its neighbor
 - English 've in *I've*, the possessive 's in *Buc-ee's*
 - French *l'* in *L'Occitane*
 - Arabic *b-* “by, with” and *w-* “and”
- why does this matter?
 - `word_tokenize` cut 's and *n't* off, and why it left *l'homme* alone because the English clitics were on its list but the French one was not

morphology

- Stephen Greenberg (1960) was an important linguistic typologist
- Greenberg (1960) studies average morphemes per word in running text
- **analytic (isolating) ↔ synthetic ↔ polysynthetic**

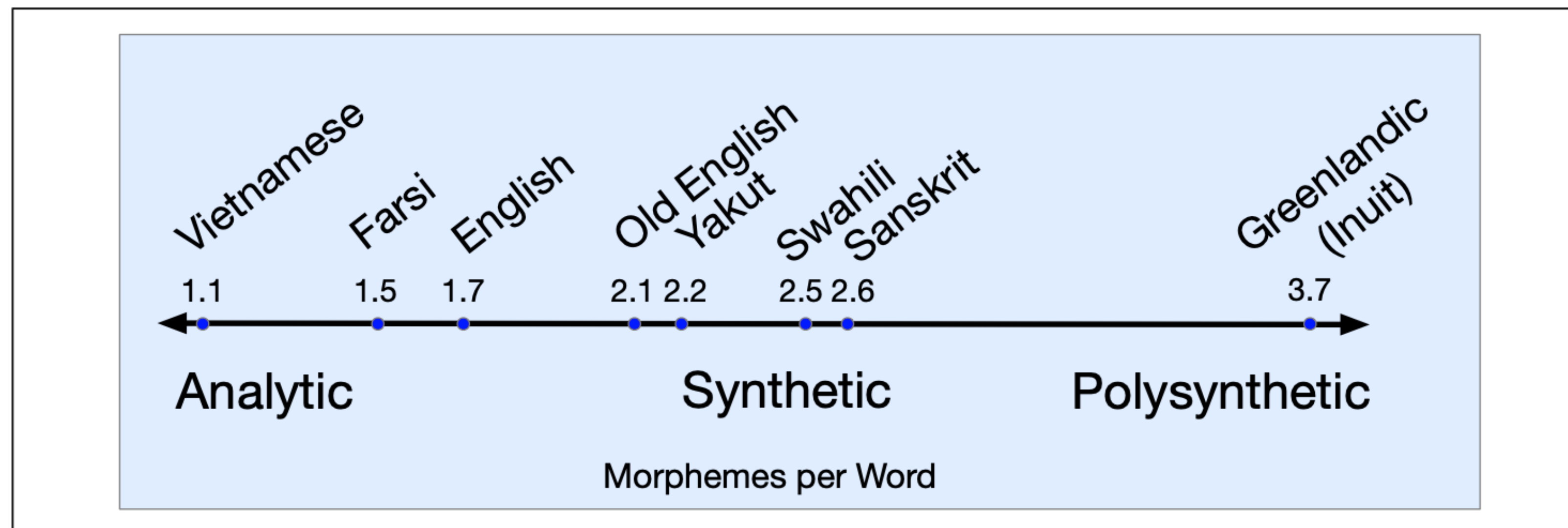


Figure 2.3 An early estimate of morphemes per word by Joseph [Greenberg \(1960\)](#).



Stephen Greenberg

morphology

- on one end of **analytic** languages we have Cantonese
- one morpheme, one syllable, one word

佢	話	全	國	最	大	間	屋	係	呢	間
keoi5	waa6	cyun4	gwok3	zeoi3	daai6	gaan1	uk1	hai6	ni1	gaan1
3SG	say	entire	country	most	big	CL	house	COP	this	CL

‘He said the biggest house in the country was this one.’

morphology

- on the end of **polysynthetic** languages we have Koryak
 - spoken on the Kamchatka peninsula

t-ə-nk'e-mejŋ-ə-jetemə-nni-k

1SG.S-E-midnight-big-E-yurt.cover-E-sew-1SG.S[PFV]

“I sewed a lot of yurt covers in the middle of a night”



morphology

- synthetic languages can be **agglutinative**: one meaning per morpheme through affixation
- e.g. Turkish

Form	Breakdown	Meaning
<i>ev</i>	ev = house or home	house or home
<i>ev-len-</i>	ev = house or home, -len = become or enter a state	get married, literally “become a home”
<i>evlen-ecek-</i>	evlen- = get married, -ecek = future	will get married
<i>evlenecek-miş-</i>	evlenecek- = will get married, -miş = apparently, reportedly, I heard	apparently will get married
<i>evlenecekmiş-siniz</i>	evlenecekmiş- = apparently will get married, -siniz = you plural or formal you	apparently you are going to get married

morphology

- synthetic languages can also be **fusional**: combining different meanings in one morpheme
- e.g. Russian
 - стол-ом means ‘with a table’
 - the ending *ом om* means instrumental + singular (INS.SG)
- English fuses too: the *-s* of *she reads* is person, number and tense at once

morphology

the computational problem

- splitting on spaces assumes a word is a morpheme
 - which is nearly true for Vietnamese and English and false for most languages
- BPE found `_re` by counting
 - but it cannot say that `_re` is a prefix, or turn *try* plus past into *tried*

morphology

the computational problem

- the goal: a grammar that does both directions on purpose

	input	output	used for
analysis	cats	cat +N +Pl	search, dictionaries, glossing, corpus queries
generation	try +V +Past	tried	paradigms, language learning tools, text generation

activity 1

morphemes per word

- write a sentence of eight or more words in a language you know, segmented with hyphens like *work-ed care-ful-ly*
- run the notebook cell to count morphemes and words from the hyphenated string and divides
- swap sentences: your partner checks and asks what each piece means
- tips: clitics like 's count as morphemes; an ending that means several things at once, like -s in *reads*, is still one morpheme; a bare root is one
- place your language on the Greenberg scale



text normalization

normalization

- **text normalization** converts the forms we have onto the forms we want to process
 - *The* and *the*
 - *cats* and *cat*
 - *lying* and *lie*
- why does it matter?
 - a search for *run* should find *running* and *ran*
 - they can be the noise that we want to eliminate (though not always)

normalization

- *Emma* has 161,600 tokens
 - we've talked about how to count them in ways such as case normalization
 - today we're going to get deeper about **lemmas** and **stems**

level	types	what merged
as written	7,535	nothing
lowercased	7,079	The, the
WordNet lemmas	5,052	cats, cat; went, go
Porter stems	4,466	respect, respectable, respectfully

normalization

case

- `.lower()` applies the English case table
 - Turkish has four letters where English has two: I ı and İ i
 - the dot on İ is left behind as a stray mark

`"İstanbul".lower()`

→ 'istanbul'

normalization

case

- `.casefold()` is lowercasing built for comparison
 - German ß unfolds to ss, so Masse ‘mass’ and Maße ‘measures’ now match
- case is a table of pairs, different for every script
 - scripts without case have no use for it

```
"Straße".lower(), "Straße".casefold()  
→ ('straße', 'strasse')
```


normalization

diacritics

- since they are added to base letters, they can have two representations
- e.g. é could be one code point, U+00E9, or e plus U+0301, a combining mark
 - the two print identically, but `==` says no and `len` disagrees
 - keyboards, transcription tools and copy-paste mix the two freely

```
a = "café"
```

```
b = "cafe\u0301"
```

```
[unicodedata.name(ch) for ch in b[3:]]
```

```
→ ['LATIN SMALL LETTER E', 'COMBINING ACUTE ACCENT']
```

```
a == b, len(a), len(b)
```

```
→ (False, 4, 5)
```

normalization

diacritics

- **NFC** composes, **NFD** decomposes

```
a = "café"
```

```
b = "cafe\u0301"
```

```
a == b
```

→ False

```
unicodedata.normalize("NFC", b) == a
```

→ True

normalization

diacritics

- HW 2 debrief
 - we can use NFD and [unicodedata.category\(\)](#) to check diacritics
 - the Greek inscription now passes the palindrome check

```
s = "Νίψον ἀνομήματα μὴ μόναν ὄψιν"  
out = ""  
for c in unicodedata.normalize("NFD", s):  
    if unicodedata.category(c) != "Mn":  
        out = out + c
```

out

→ 'Νιψον ανομηματα μη μοναν οψιν'

normalization

diacritics

- diacritic stripping is a decision to make
 - Spanish: ñ is a letter of the alphabet; año and ano are different words
 - Vietnamese: the marks carry tone; ma, má, mà, mã, mạ, mả are six words
- there is **no script-neutral cleaning**

normalization

stemmer

- a **stemmer** strips suffixes by rule, with no dictionary
 - -s, -ing, -ution come off in ordered steps
 - Porter (1980) has about sixty rules
- a stem **need not** be a word
 - strang, distribut: it only has to be consistent, so distributing and distribution land on the same string

```
raw = "strange women lying in ponds distributing"  
porter = nltk.PorterStemmer()  
[porter.stem(w) for w in raw.split()]  
→ ['strang', 'women', 'lie', 'in', 'pond', 'distribut']
```

normalization

stemmer

- nltk ships several stemmers
- stemming is not a well-defined process; it is whatever a rule set does
 - pick the one that suits the application and be consistent

word	Porter	Lancaster
lying	lie	lying
women	women	wom
ceremony	ceremoni	ceremony
distributing	distribut	distribut
power	power	pow

normalization

lemmatizer

- **WordNet** is a dictionary organized by meaning (semantics), not by spelling
 - 117,659 *synsets*, sets of words that share one sense
 - woman.n.01 is woman, adult_female; every entry is a **lemma** with a part of speech
- the **lemmatizer** looks a candidate up here
 - lie has ten entries, three nouns and seven verbs, so lie as a verb exists and lying can become it

```
from nltk.corpus import wordnet as wn
```

```
wn.synsets("lie")[:4]
```

```
→ [Synset('lie.n.01'), Synset('lie.n.02'), Synset('lie.n.03'), Synset('lie.v.01')]
```

```
wn.synset("lie.v.01").definition()
```

```
→ 'be located or situated somewhere; occupy a certain position'
```

normalization

lemmatizer

- a lemma is the dictionary headword
 - a lemmatizer is like looking up a dictionary
 - cats → cat, women → woman, went → go
- the WordNet lemmatizer removes an affix only if the result is in its dictionary
 - so it gets women which no rule could and leaves lying alone until told it is a verb

```
wnl = nltk.WordNetLemmatizer()
```

```
[wnl.lemmatize(w) for w in ["women", "lying", "ponds"]]
```

```
→ ['woman', 'lying', 'pond']
```

```
wnl.lemmatize("lying", "v")
```

```
→ 'lie'
```

normalization

irregulars

- **irregular forms come from a lookup table**
 - WordNet keeps exception lists: about 2,000 nouns, 2,400 verbs, 1,500 adjectives
 - handles situations such as foot/fee, went/go, etc.

```
wnl.lemmatize("feet"), wnl.lemmatize("went", "v"),  
wnl.lemmatize("better", "a")
```

→ ('foot', 'go', 'good')

```
porter.stem("feet"), porter.stem("went")
```

→ ('feet', 'went')

normalization

stemmer vs. lemmatizer

- a lemmatizer is a dictionary lookup first and suffix rules second; a stemmer is rules only
- in linguistics a type usually means a lemma type; in NLP it usually means a word type

	stemmer	lemmatizer
knows	rules	a dictionary, plus rules
output	a stem, maybe not a word	a headword
speed	fast	slow
good for	search indexes	vocabularies, glosses
fails on	women; morning → morn	unknown words; untagged verbs

activity 2

- each of you picks a suffix: -ness, -ies, -ing, -ly or -ed
- pull every Emma word that ends in it, then run `porter.stem` and `wnl.lemmatize` on the list
- find one word where the stemmer is wrong and one where the lemmatizer is wrong
- tips
 - `set()` for the types
 - `wnl.lemmatize(w, "v")` for verbs
- share the two words and say which tool you would trust for that suffix



next time

- Thursday
 - evaluation and edit distance
- HW 3 to be released Thursday